

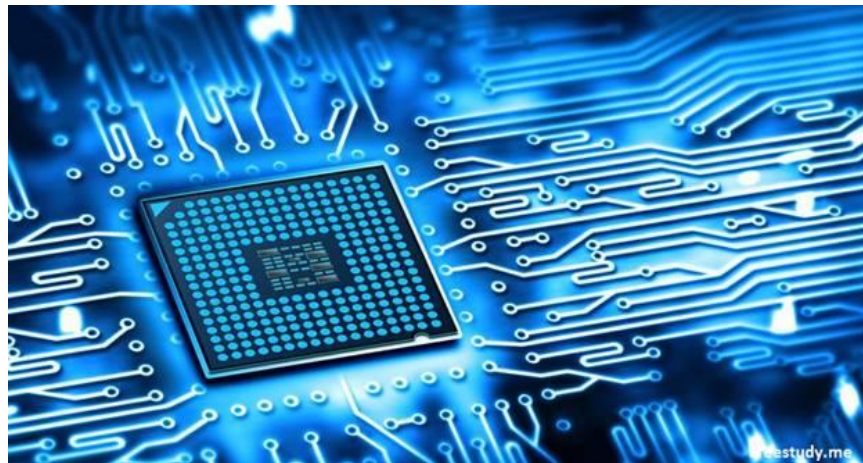
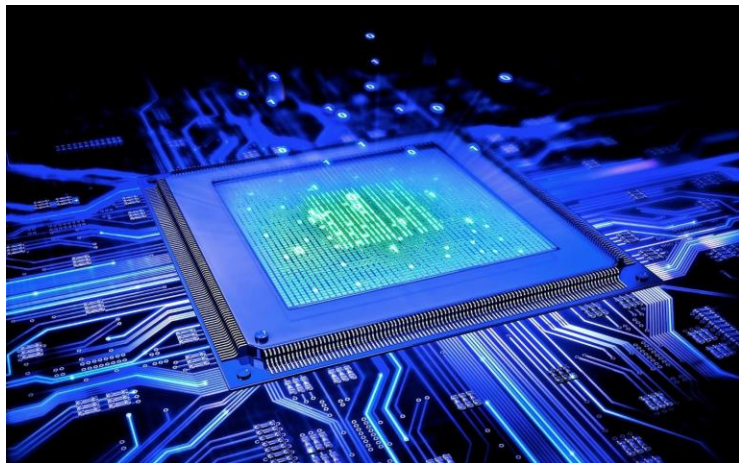
ІТМО

**~~Программирование ПЛИС~~
Проектирование на ПЛИС**

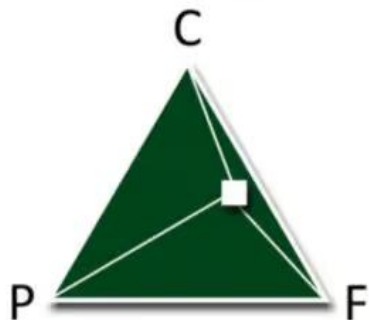
Вводная

Власов Сергей Михайлович

Программируемая логическая интегральная схема (ПЛИС, англ. *programmable logic device, PLD*)

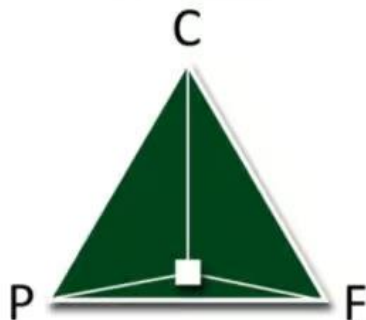
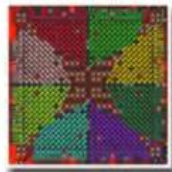


Traditional CPU



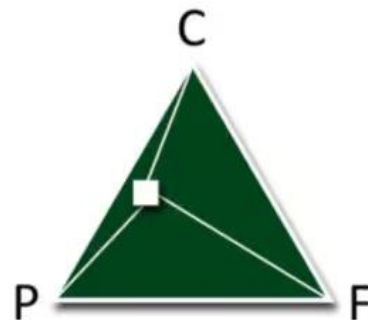
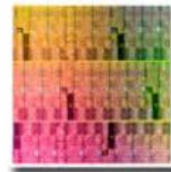
COST ~
PERFORMANCE
FLEXIBILITY

FPGA



COST
PERFORMANCE
FLEXIBILITY

Network ASIC



COST
PERFORMANCE
FLEXIBILITY



	ASIC	ASSP	SoC	FPGA
Design Style	Fully Custom	Fixed Function	Mix of Custom and IP	Configurable Fabric
Process	45nm – 7nm node	28nm – 180nm	5nm – 65nm	16nm – 28nm
Typical Gate Count	10M – 5B	50K – 1M	5M – 2B	100K – 5M
Development Time	24+ months	6-12 months	12-36 months	3-12 months
Mask Cost	\$2M – \$5M	\$500K	\$1M – \$3M	Minimal
Unit Cost	\$1 – \$500+	\$2 – \$100	\$10 – \$150	\$5 – \$100
Design Expertise	Full custom IC	System integration	Processor, digital/analog/RF IC	Hardware description language
Applications	High-volume consumer electronics, network systems, computing	Broad market horizontal functions	Consumer devices, edge servers, automotive, etc.	Aerospace/defense, instrumentation, networking, acceleration



Типы ПЛИС

- PAL - *programmable array logic*
- GAL - *generic array logic*
- CPLD - *complex programmable logic device*
- FPGA - *field-programmable gate array*



Некоторые производители

- Achronix
- Actel -> Microsemi
- Altera -> Intel
- Atmel -> Microchip Technology
- Lattice semiconductor
- Xilinx - AMD
- GoWin Semiconductor





CPLD

VS

FPGA



Power

As soon as you switch on the power it gets started.

Since FPGA has to load configuration data from external ROM and set up the fabric before it can start functioning, there is a time delay between power ON and FPGA starts working.



Power Off

Non-volatile. CPLDs stay programmed and retain their circuit after powering down.

FPGAs go blank as soon as powered off. It uses SRAM-based configuration storage. The contents of the memory are lost as soon as power is off.



Timing Analysis

Deterministic Timing Analysis. Since CPLDs are comparatively simpler than FPGAs, and the number of interconnects is less, the timing analysis can be done much more easily.

The size and complexity of FPGA logic can be humongous and opens up the possibility of less deterministic signal routing and thus causing complicated timing scenarios. However, implementation tools provided by FPGA vendors have mechanisms to assist in achieving deterministic timing.

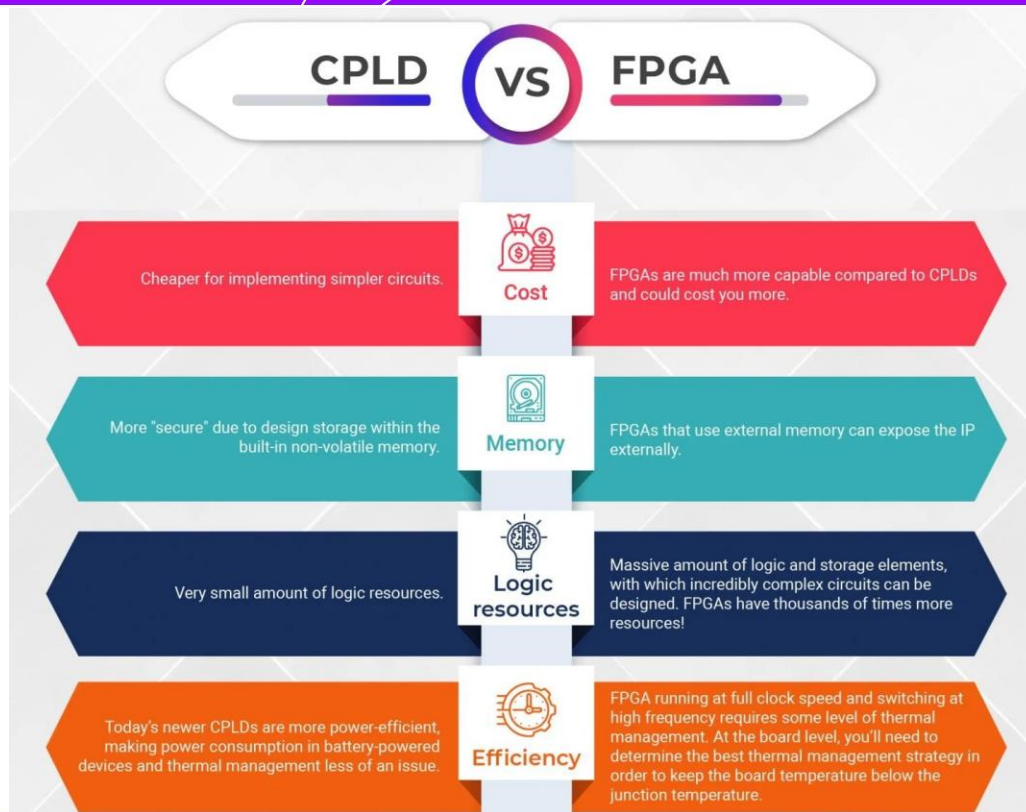


Power consumption

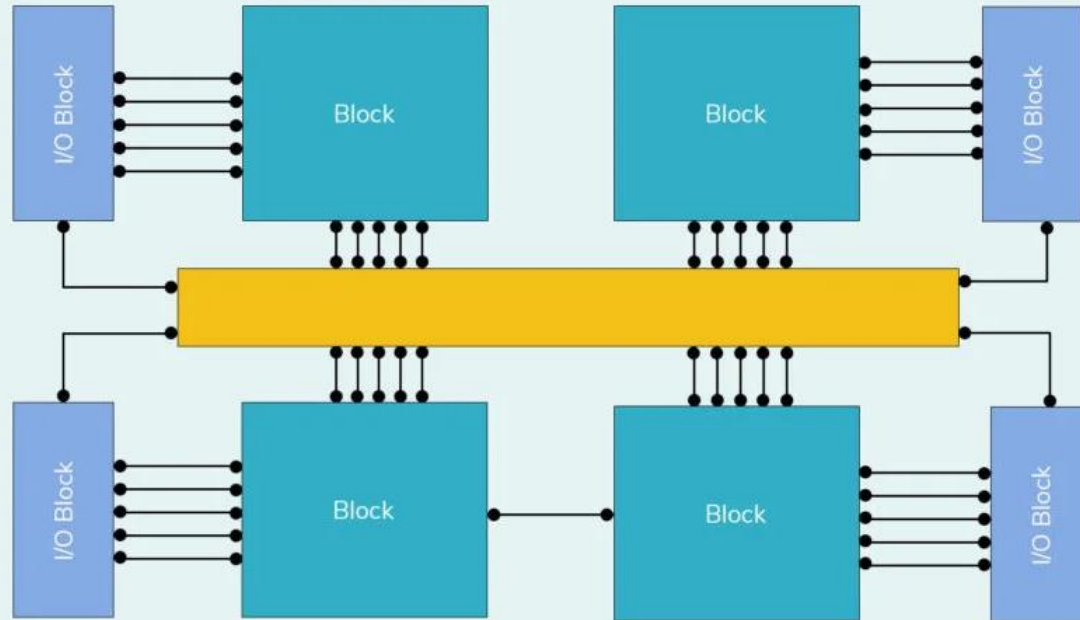
Lower idle power consumption. Newer CPLDs such as CoolRunner-II use around 50 uA in idle conditions.

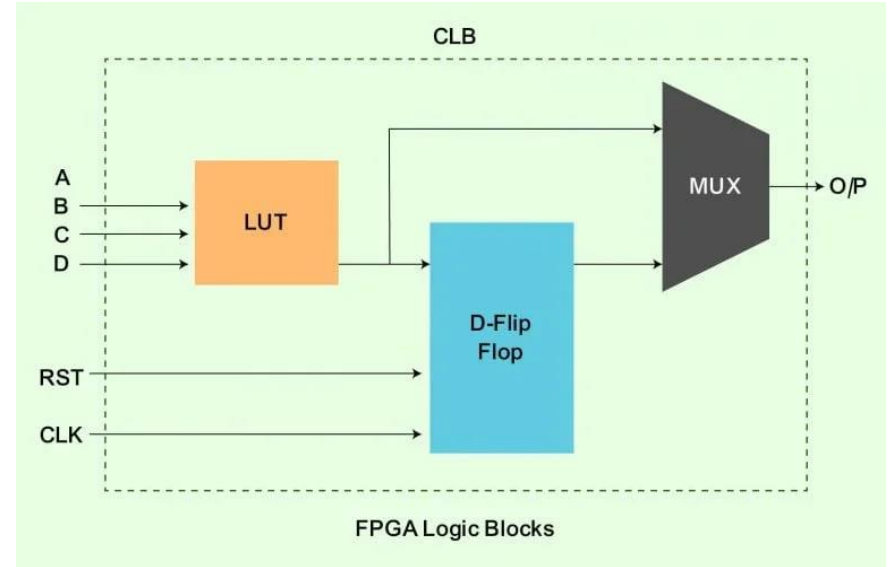
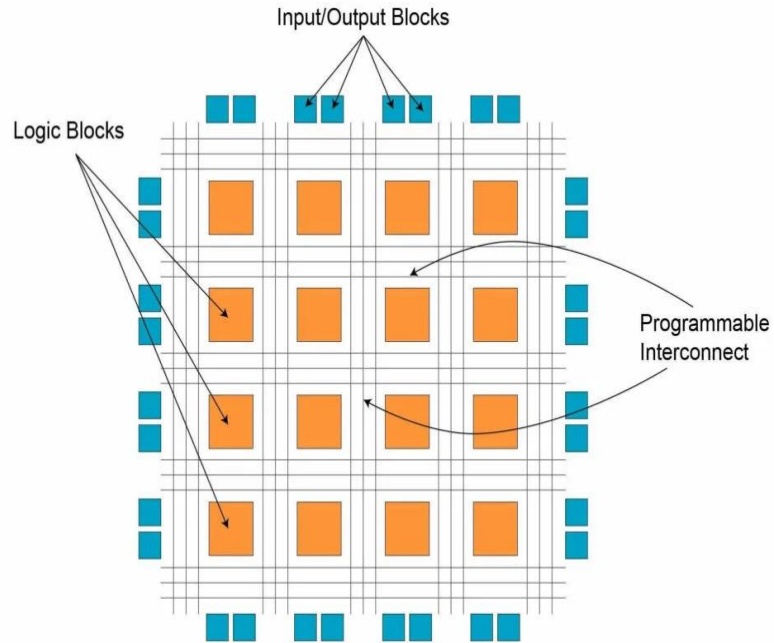
Relatively higher idle power consumption.





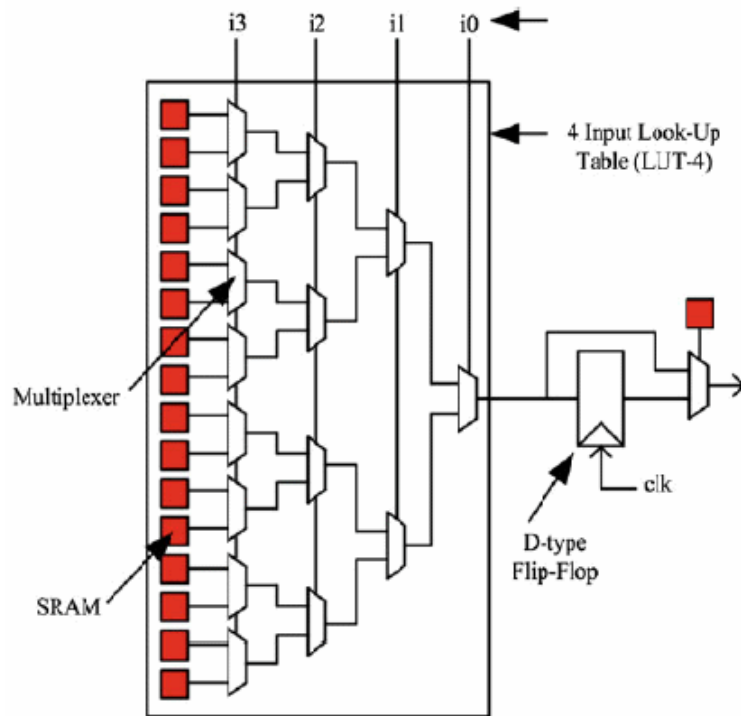
Block Diagram of CPLD





LUT – LookUp Table

Конфигурируемые логические блоки

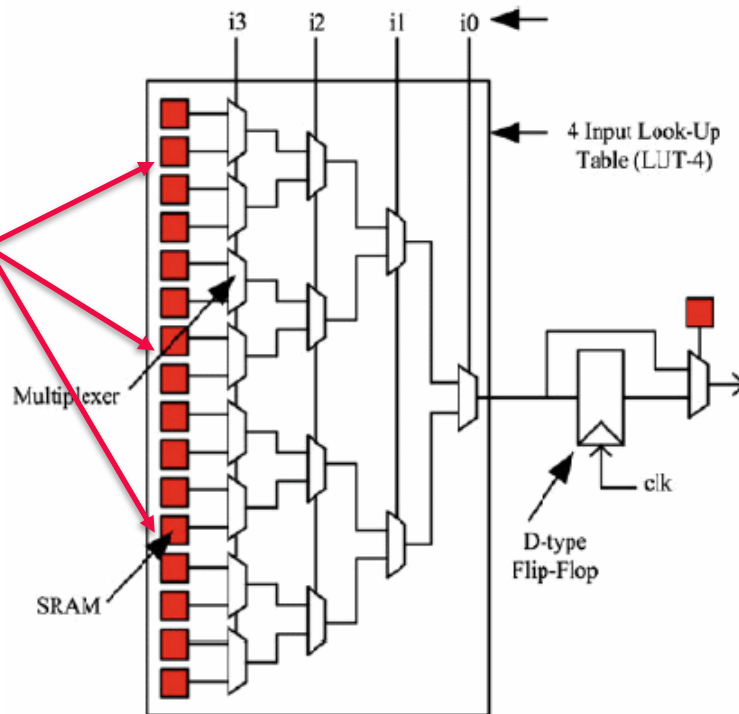


LUT – это Look-Up Table, таблица преобразования.

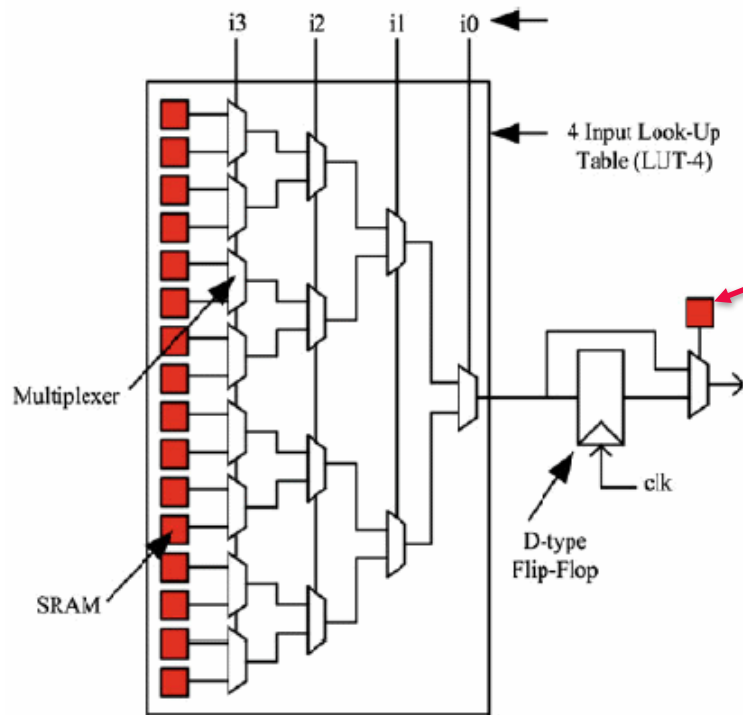
4х битовому числу на входе (i3-i0) ставится соответствие 1 битовый результат.

Конфигурируемые логические блоки

Программируемое значение



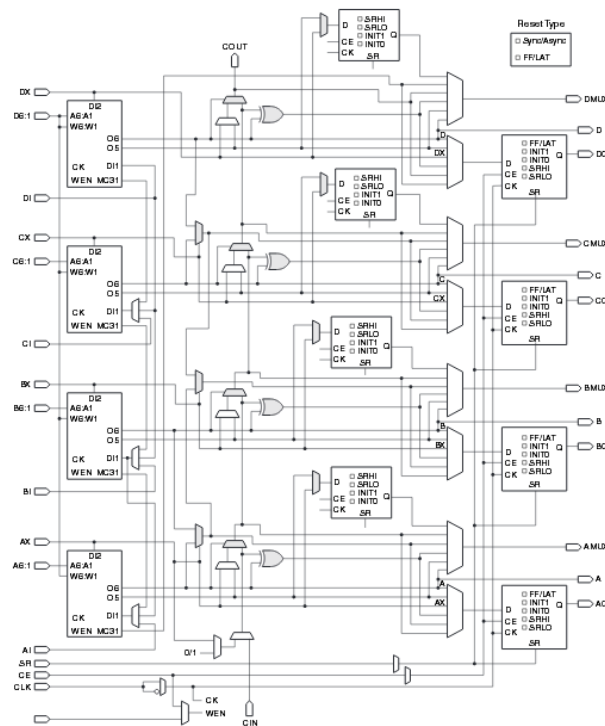
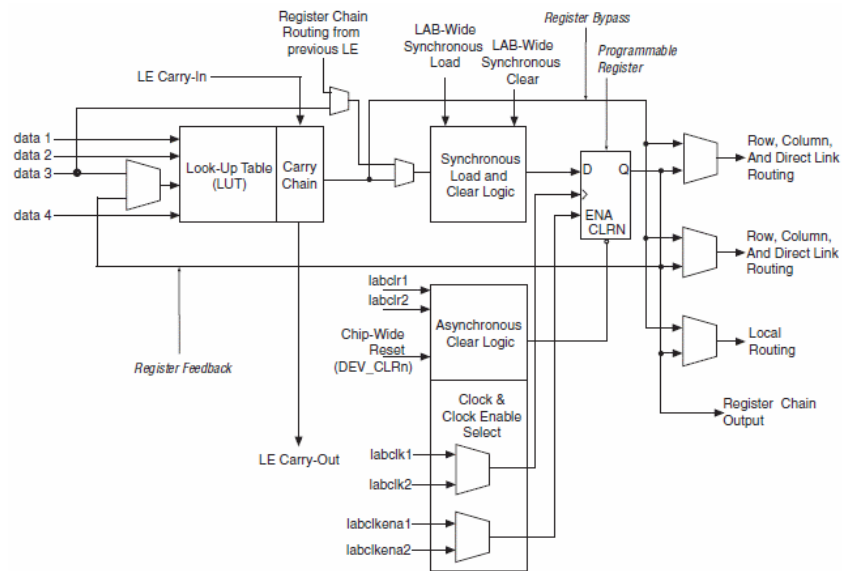
Конфигурируемые логические блоки



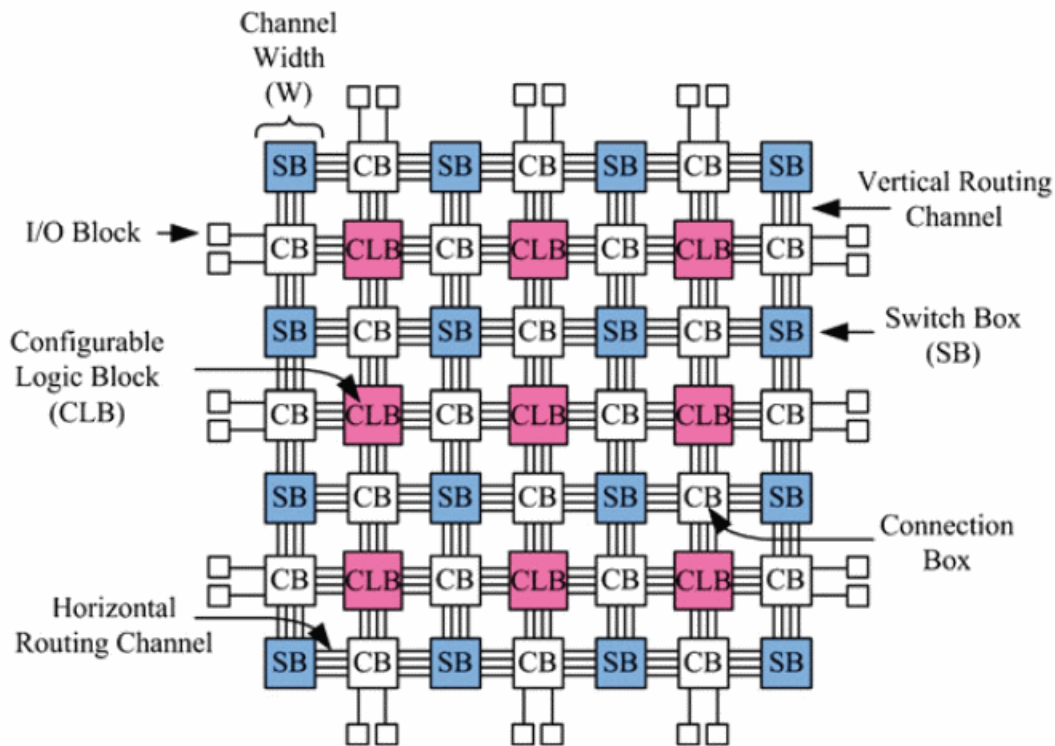
включение использование триггера



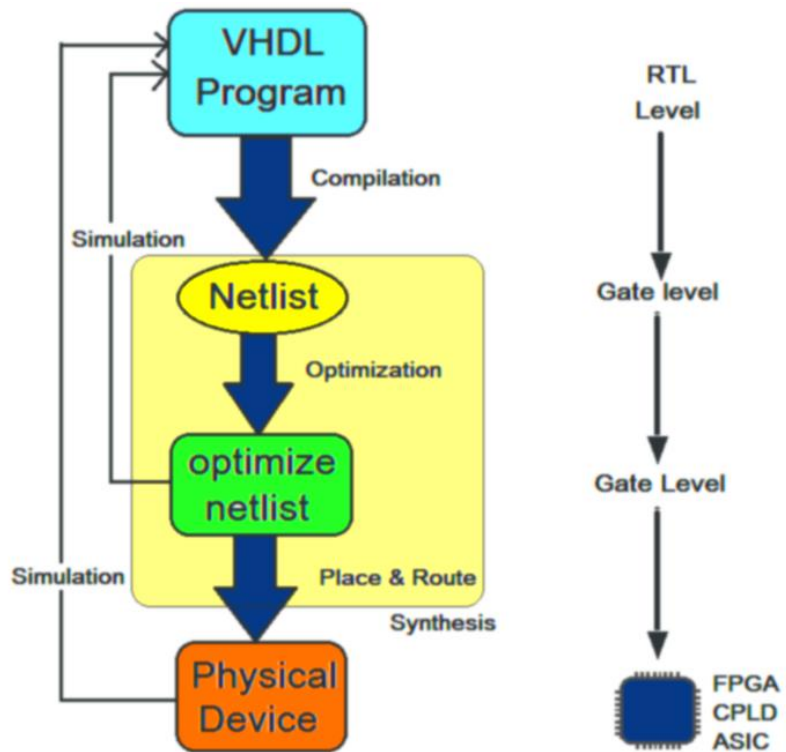
Конфигурируемые логические блоки

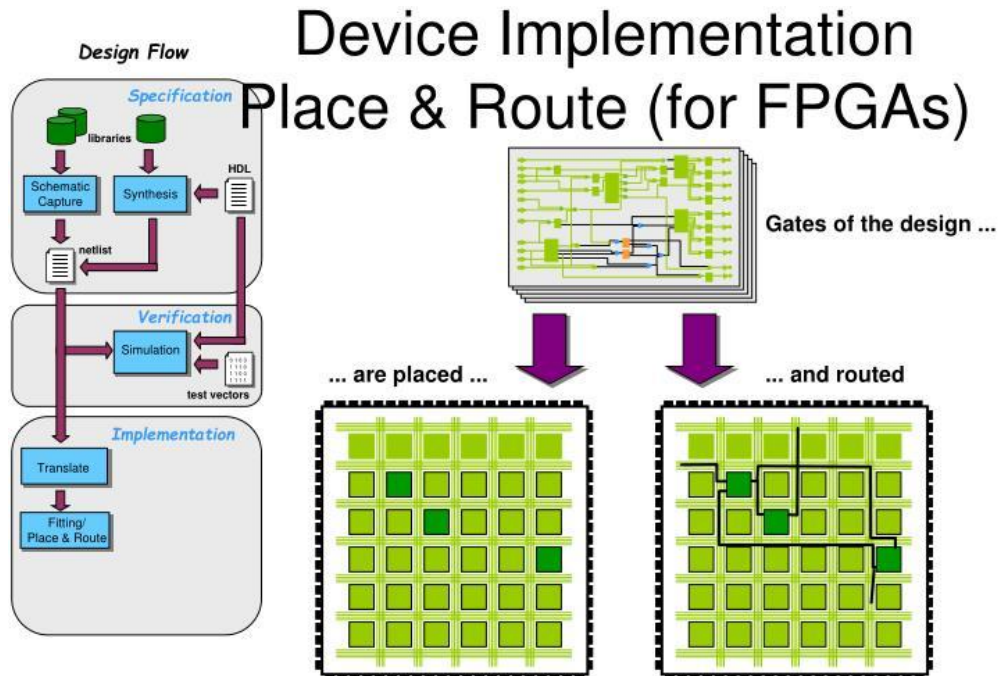


Конфигурируемые логические блоки



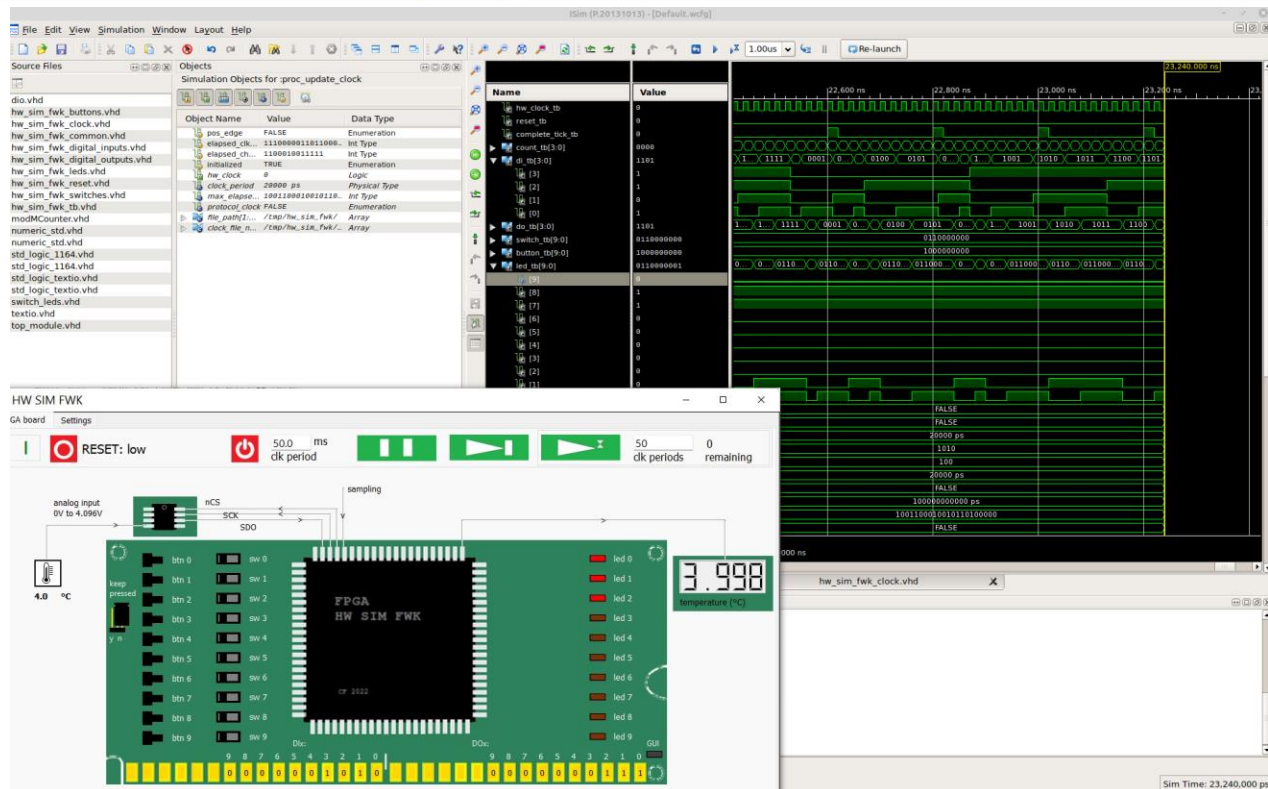
Разработка проекта для ПЛИС





Разработка проекта для ПЛИС

ИТМО



Языки программирования

Hardware description language



VHDL	VERILOG	SYSTEM VERILOG
V-VHSIC Very high-speed IC HDL- hardware description language	Verilog- Verify logic -HDL Hardware description language	System Verilog-HVL Hardware verification language
Case Insensitive	Case sensitive	Case sensitive
static	static	Static and dynamic
It support Library management	It does not support Library management	It support Library management
Assigning of <u>four bit</u> register to an 8-bit register is a compilation error	Assigning of <u>four bit</u> register to an 8-bit register is not a compilation error	Assigning of <u>four bit</u> register to an 8-bit register is not compilation error
It does not support OOPS features	It does not support OOPS features	It support OOPS features
It does not have functional coverage	It does not have functional coverage	It supports functional coverage
It does not support assertion-based verification	It does not support assertion-based verification	It support assertion-based verification
Mainly used for Design and Verification of FPGA'S	Mainly used for Design of ASICS AND SOCS	Mainly used for Design and Verification
It does not support constraints random driven verification	It does not support constraints random driven verification	It support constraints random driven verification
It does not support PLI	It support PLI	It support PLI

Verilog HDL (Hardware Description Language) - это язык текстового описания аппаратуры.



Плюсы языка

- C-like syntax
- Более компактный код по сравнению с VHDL
- Block commenting
- Нет необходимости в создании сложных экземпляров компонентов, как в VHDL

```
def calculate_average(numbers):  
    """  
    This function calculates the average of a list of numbers.  
  
    Args:  
        numbers (list): A list of numeric values.  
  
    Returns:  
        float: The average of the numbers in the list.  
    """  
    total = sum(numbers)  
    average = total / len(numbers)  
    return average
```

Пример block commenting в Python
для понимания

Hardware description language

■ Verilog HDL vs. VHDL

```
library ieee;
use ieee.std_logic_1164.all;

entity ff is
port(
  reset, clk, d :in  std_logic;
  q             :out std_logic);
end ff;

architecture logic of ff is
begin

process(reset, clk)
begin
if (reset='1') then
  q <= '0';
elsif (clk='1' and clk'event) then
  q <= d;
end if;
end process;

end logic;
```

```
module ff(reset, clk, d, q)
input  reset, clk, d;
output q;

always @(posedge clk or posedge reset)
begin
  if (reset)
    q <= 1'b0;
  else
    q <= d;
  end if;
end module;
```

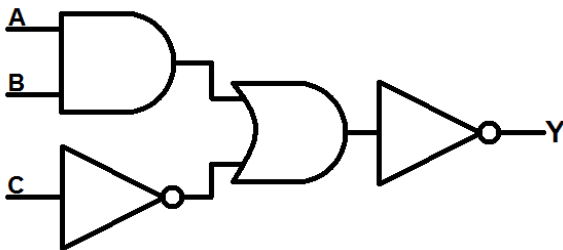
Примеры решение одной и той же задачи на VHDL(слева) и Verilog(справа)



Комбинационные схемы – цифровые схемы выходы, которых зависят от текущего входа.

Такие схемы состоят только из комбинационных вентилях и цепей, и желательно должны быть составлены по следующим правилам:

- Каждый элемент комбинационный вентиль
- Каждое соединение это или вход, или соединен с единственным выходом
- Не содержит циклических путей



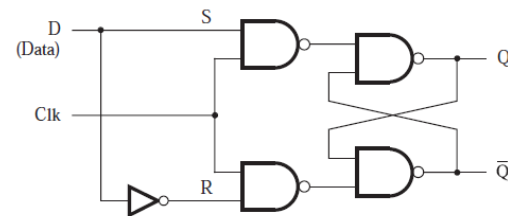
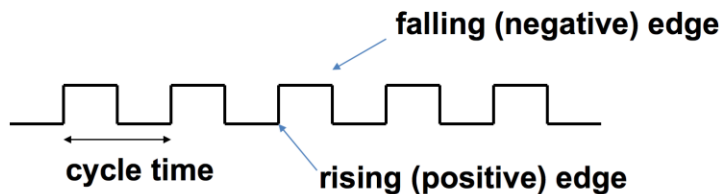
Пример комбинационной схемы



Пример не комбинационной схемы

Защелки/Триггеры/Регистры

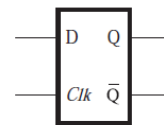
Триггеры(flip-flops) и защелки(latches) – это бистабильные элементы (элементы с двумя стабильными состояниями в нашем случае 1 и 0) с тактовым входом. Отличие триггеров от защелок обычно заключается в том, что триггеры меняют состояние по фронту (обычно возрастающему)



(a) Circuit

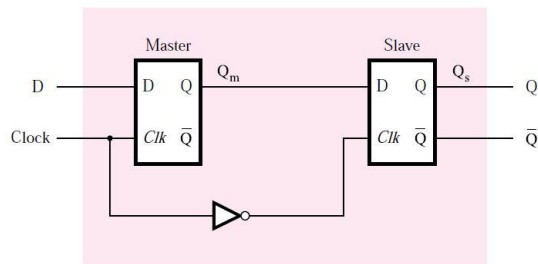
Clk	D	$Q(t+1)$
0	x	$Q(t)$
1	0	0
1	1	1

(b) Characteristic table



(c) Graphical symbol

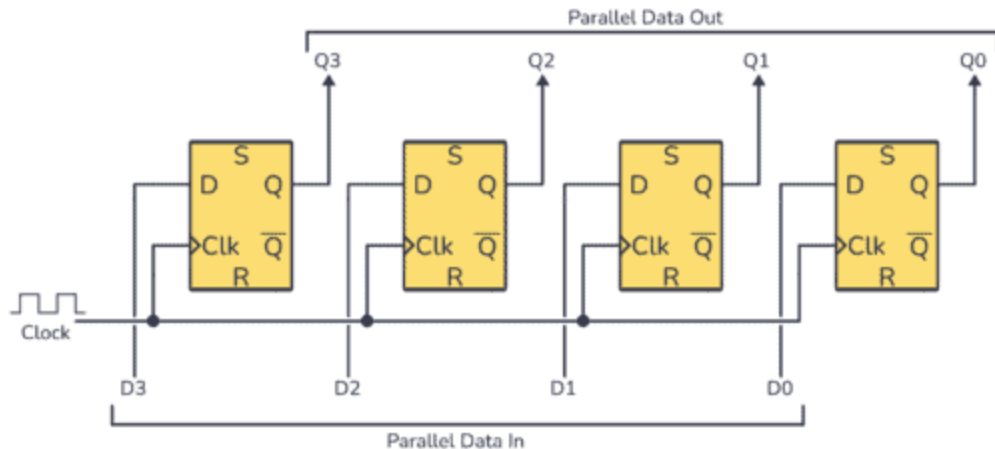
D-защелка



D-триггер
активирующийся по
падающему фронту

Защелки/Триггеры/Регистры

N-разрядный регистр (N-digit register) – это набор из N триггеров с общим тактовым сигналом. Можно считать, что триггер частный случай регистра.

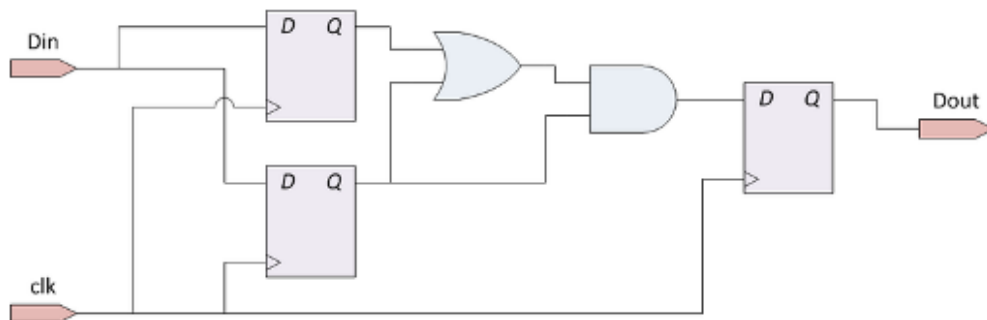


4-х битный регистр на D-триггерах

Синхронные (последовательные) цифровые схемы состоят из

- комбинационных вентилей
- цепей
- регистров

В синхронной схеме есть единственный сигнал синхронизации, который управляет всеми элементами памяти (регистрами).

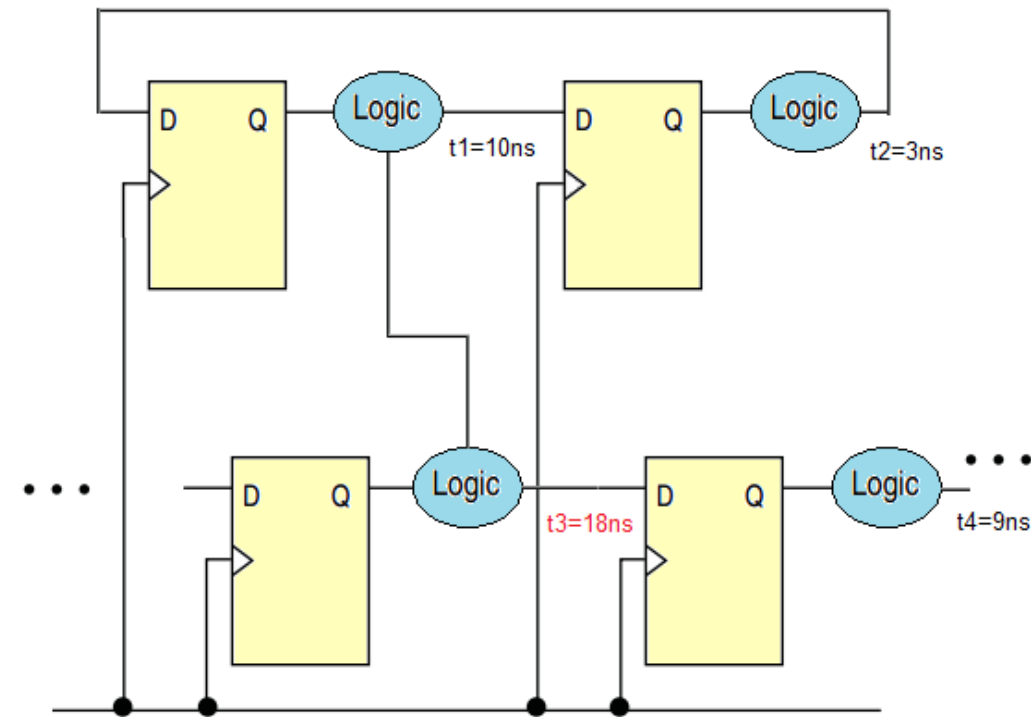


Формально, синхронная схема - это любая цифровая схема, которая придерживается следующих правил:

- В схеме существует единственная цепь **clk**, по которой распространяется сигнал синхронизации (тактовый сигнал, clock signal).
- Цепь должна состоять только из регистров и комбинационной логики, при этом должен быть хотя бы один регистр, иначе цепь можно считать комбинационной (если выход зависит только от текущего входа)
- Если существует обратная связь (циклический путь) она должна обязательно содержать регистр.

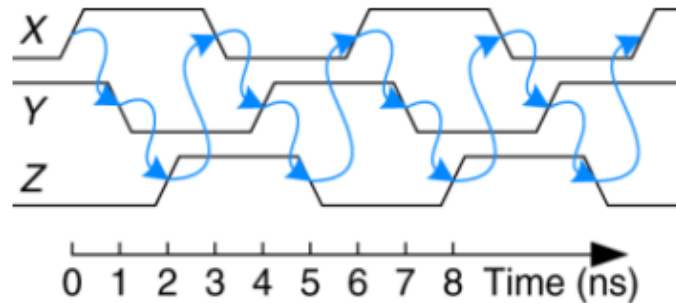
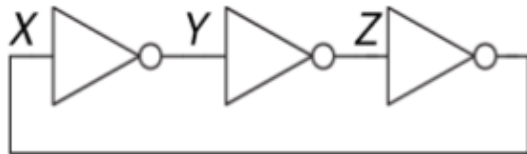


Синхронные схемы



CLK Fmax = $1/18\text{ns} = 55\text{MHz}$

Пример неустойчивой схемы

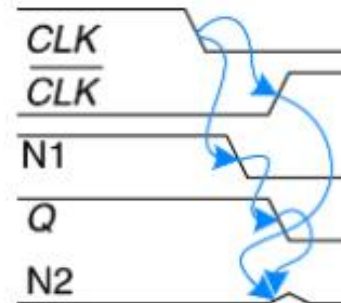
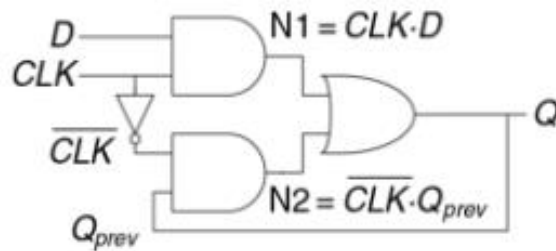


Пример гонок в последовательных схемах

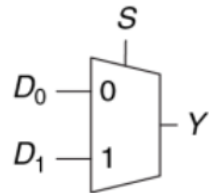


CLK	D	Q_{prev}	Q
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	0
1	1	0	1
1	1	1	1

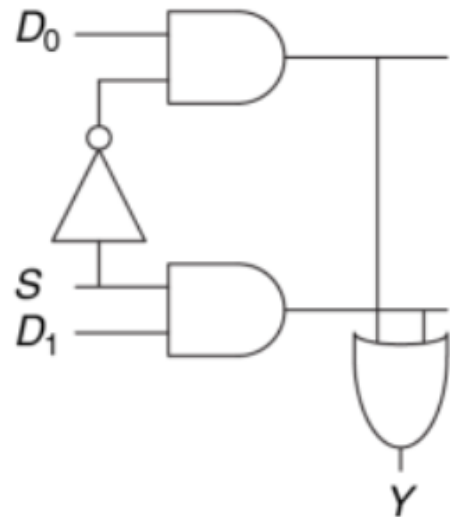
$$Q = CLK \cdot D + \overline{CLK} \cdot Q_{prev}$$



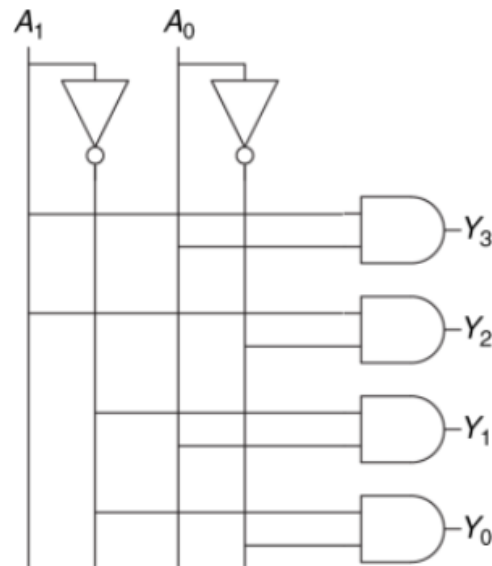
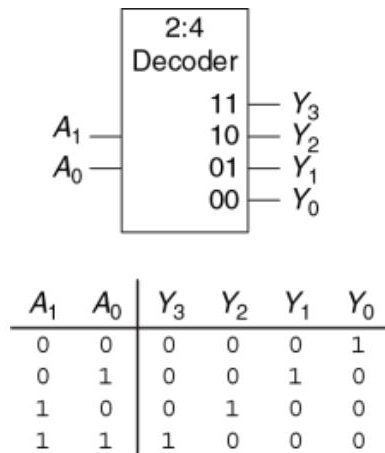
Мультиплексоры



S	D_1	D_0	Y
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	0
1	1	0	1
1	1	1	1

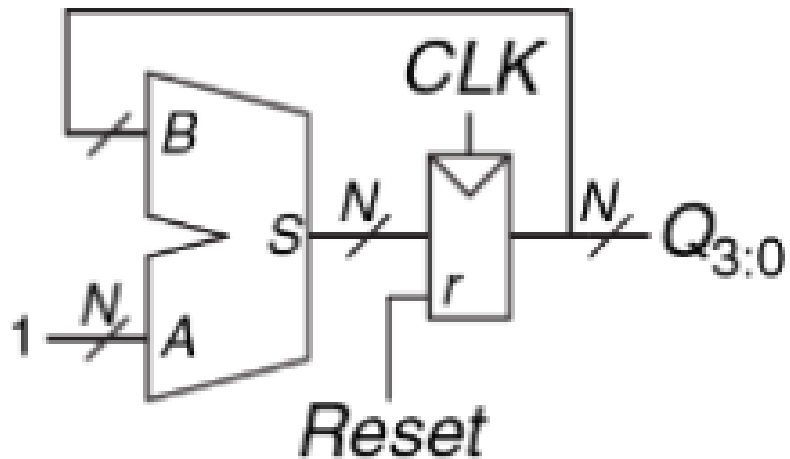


Дешифратор



Важные комбинационных блока

Счетчик.



Важные комбинационных блока

Регистр сдвига

